



CSE 232 Systems Programming Term Project Report

TextEditor with Garbage Collection

**Ege Huriel
Cihat Özlük
Musab Tunç
Ferhat Canşı
Atakan Naktürk**

Assigned by Asst. Prof. Dr. Esin Onbaşıoğlu
June 08, 2026

Contents

Contents	2
1. Introduction.....	3
2. Project Objectives.....	3
3. Data Structure Design	4
3.1 The Node Structure	4
3.2 Pointer Invariants	4
3.3 Invariant Summary.....	4
4. Module Architecture	5
4.1 editor.c — Buffer Core	6
4.2 operations.c — Editing Operations.....	6
4.3 garbageCollection.c — Memory Compaction	6
4.4 ui.c — ncurses User Interface	7
4.5 main.c — Command Loop and ncurses Integration.....	7
5. Garbage Collection — Detailed Analysis	8
5.1 The Problem of Fragmentation	8
5.2 The Collection Algorithm.....	8
5.3 Buffer State Example.....	8
5.4 Trigger Policy	9
6. User Interface.....	10
6.1 Screen Layout	10
6.2 ncurses Initialization	10
6.2 Command Summary	10
7. Build System.....	11
7.1 Directory Structure.....	11
7.2 Makefile.....	11
8. Test Cases and Expected Behavior.....	12
8.1 Load and Save	12
8.2 Insertion.....	12
8.3 Deletion	13
8.4 Garbage Collection	13
8.5 Automatic GC Trigger	14
9. Team Contributions and Responsibilities	15
10. Conclusion	16
References.....	16

1. Introduction

This report presents the design, implementation and testing of a simple text editor with garbage collection, developed as the term project for CSE 232 - Systems Programming at Yeditepe University. The editor is written entirely in C programming language, compiled with GCC and uses ncurses library to provide a fullscreen, cursor driven terminal user interface.

The core problem this project addresses is a classical topic in systems programming: managing a finite, statically allocated memory buffer in a way that supports dynamic editing operations — insertion, deletion and modification of text lines — without resorting to dynamic memory allocation (malloc/free). Instead, the editor employs a doubly linked list overlaid on a fixed-size array and reclaims fragmented space through periodic garbage collection passes.

The project is organized into five source modules that cleanly separate concerns: the data structure and file I/O layer (*editor.c*), the editing operations (*operations.c*), the garbage collector (*garbageCollection.c*), the ncurses user interface (*ui.c*) and the main command loop (*main.c*). All five modules are fully implemented and integrated.

At its core, its a terminal-based text editor written in C. The editor manages a text buffer in memory using a doubly linked list structure and performs garbage collection to reclaim deleted entries.

2. Project Objectives

The project was assigned with the following primary goals:

- Implement a memory-resident text buffer using a statically allocated array of node structures, managed as a doubly linked list.
- Provide a command-driven user interface supporting the operations E (edit), I (insert), D (delete), R (replace), P (print), S (save), G (garbage collection) and Q (quit).
- Integrate the ncurses library to enable cursor navigation across lines and characters, making the editor interactive rather than purely command-line driven.
- Implement a garbage collection routine that compacts the text buffer by eliminating slots orphaned by deletion and trigger this routine both manually (via the G command) and automatically (after every 10 mutating operations or when the buffer is full)/
- Structure the codebase in a modular and maintainable way, with a dedicated header file per module and a Makefile build system.

3. Data Structure Design

3.1 The Node Structure

The fundamental data structure is a doubly linked list embedded within a statically allocated array. Each element of the array is a node:

```
typedef struct node {
    char statement[MAX_CHARS]; /* line content, max 40 characters */
    int next;                  /* index of the next valid line */
    int prev;                  /* index of the previous valid line */
} Node;

Node textbuffer[MAX_LINES]; /* global array - max 100 lines */
int head;                   /* index of the first valid line */
int tail;                   /* index of the last valid line */
int freeIndex;               /* index of the first free slot */
```

The sentinel value NIL (defined as -1) is used wherever a pointer would be null in a conventional linked list. All five globals — `textbuffer`, `head`, `tail`, `freeIndex` and `currentFilename` — are declared in `editor.c` and exposed to other transition units via external declarations in `editor.h`

3.2 Pointer Invariants

A crucial design property is the distinction between the logical order of lines (the traversal order defined by `next/prev` links) and the physical order of slots in the array (sequential indices 0 through 99). After a sequence of insertions and deletions, these two orders diverge: logically the text may read “aaa → ccc → ddd → xxx” while physically the slots may be occupied in the order 0, 2, 3, 4. The slot at physical index 1 (formerly “bbb”) has been unlinked and is wasted until garbage collection runs.

This separation is intentional and is what gives the garbage collector its role: it restores the invariant that all valid data occupies the first `freeIndex` slots of the array, maximizing available space for new insertions.

3.3 Invariant Summary

At all times the following invariants are maintained:

- `textbuffer[head].prev == NIL`
- `textbuffer[tail].next == NIL`
- For every valid node `i` in the chain: `textbuffer[textbuffer[i].next].prev == i` and `textbuffer[textbuffer[i].prev].next == i`
- `freeIndex` is the smallest index such that `textbuffer[freeIndex]` has never been written to or has been returned to a clean state by garbage collection

Delete nodes have their own `next` and `prev` fields set to NIL (as implemented in `deleteNode()`), but their statement content left intact until overwritten by a future insertion or cleared by garbage collection. This lazy deletion strategy is intentional, it avoids unnecessary `memset` calls on each deletion and delegates cleanup to the GC pass.

4. Module Architecture

The project is structured into four source files under the `src/` directory, each with a corresponding header inside `include/`. The build is managed by a Makefile, that compiles each translation unit separately into object files under `build/` and links them into the final editor executable.

Function Signature	Source File	Description
<code>void edit(char *filename)</code>	<code>editor.c</code>	Opens a <code>.txt</code> file, loads lines into <code>textbuffer[]</code> , sets <code>head/tail/freeIndex</code>
<code>void save(void)</code>	<code>editor.c</code>	Traverses the linked list from <code>head</code> , writes each line to the open file
<code>void clearBuffer(void)</code>	<code>editor.c</code>	Resets all 100 slots and global pointers to initial state
<code>void stripNewline(char[])</code>	<code>editor.c</code>	Removes trailing <code>\n</code> or <code>\r</code> from a string read via <code>fgets</code>
<code>int insert(int bufIndex)</code>	<code>operations.c</code>	Reads a new line from the user via <code>ncurses</code> and inserts it after the node at buffer index <code>bufIndex</code>
<code>int deleteNode(int bufIndex)</code>	<code>operations.c</code>	Unlinks the node at buffer index <code>bufIndex</code> from the doubly linked list
<code>int replace(int bufIndex, int col)</code>	<code>operations.c</code>	Overwrites a single character at buffer index <code>bufIndex</code> , column <code>col</code> , via <code>ncurses</code> input
<code>int garbageCollection(void)</code>	<code>garbageCollection.c</code>	Compacts all valid nodes to the front of <code>textbuffer[]</code>
<code>void maybeAutoGC(void)</code>	<code>garbageCollection.c</code>	Increments <code>gcCounter</code> ; triggers GC when counter reaches 10 or buffer is full
<code>void ui_init(void)</code>	<code>ui.c</code>	Initialises <code>ncurses</code> : <code>initscr</code> , <code>raw</code> , <code>noecho</code> , <code>keypad</code> , <code>curs_set</code>
<code>void ui_cleanup(void)</code>	<code>ui.c</code>	Calls <code>endwin()</code> to restore terminal state on exit
<code>void print_buffer(void)</code>	<code>ui.c</code>	Redraws header bar, separator, all text lines with selection highlight, and restores cursor
<code>int cursorLine(void)</code>	<code>ui.c</code>	Translates cursor Y coordinate to <code>textbuffer[]</code> buffer index
<code>int cursorLineNth(void)</code>	<code>ui.c</code>	Translates cursor Y coordinate to logical list position (0-based)
<code>int cursorChar(void)</code>	<code>ui.c</code>	Translates cursor X coordinate to character position within the selected line
<code>void ui_move_up/down/left/right(void)</code>	<code>ui.c</code>	Moves the <code>ncurses</code> cursor one step and refreshes the display
<code>void ui_status(const char *msg)</code>	<code>ui.c</code>	Writes a message to the bottom status bar with reverse-video styling

4.1 editor.c — Buffer Core

This module owns all global state. The *edit()* function uses *fgets()* in a loop to read lines from the file, strips trailing newline characters with *stripNewline()*, copies each line into *textbuffer[i].statement* and wires the *prev/next* pointers sequentially. After loading, *head* is set to 0, *tail* to *i-1* and *freeIndex* to *i*. A call to *clearBuffer()* at the start of *edit()* ensures that any previously loaded document is discarded cleanly before a new file is opened.

The *save()* function traverses the chain from *head* to *tail* following the *next* links, writing each statement followed by newline character to the file opened for writing. The filename used is *filename*, which is set by *edit()* at load time.

4.2 operations.c — Editing Operations

This module implements the three structural editing operations. A static helper function *nthNode(n)* traverses the linked list from *head* to find the node at logical position *n* (zero-indexed), returning *NIL* if the list is shorter than *n* elements. This function is used internally by all three public functions.

The *insert()* function handles three distinct cases: inserting into an empty list (the new node becomes both *head* and *tail*), inserting at position 0 (the new node becomes the new *head*), and inserting in the middle or at the end (the new node is spliced into the chain between the node at index *-1* and the node at index, or appended after *tail* if index is out of range). The new node is always placed at *textbuffer[freeIndex]* and *freeIndex* is incremented.

The *deleteNode()* function unlinks the target node by updating the *prev's next* and *next's prev* pointers, promoting the next node to *head* or the *prev* node to *tail* as appropriate. The target nodes own *next* and *prev* are then set to *NIL*, signaling to the garbage collector that the slot is orphaned.

The *replace()* function uses *nthNode()* to locate the target row and directly overwrites the character at the specified column index within *textBuffer[target].statement*.

4.3 garbageCollection.c — Memory Compaction

The garbage collection module is the most algorithmically focused component of the project. The *garbageCollection()* function performs a two-phase operation: first, it traverses the linked list from *head* to *tail* via *next* pointers, copying each valid statement into a local temporary array (*Node temp[MAX_LINES]*); second, it calls *clearBuffer()* to wipe the global state, then copies the collected statements back into *textbuffer[0..count-1]* with fresh sequential *prev/next* links.

The effect is that all previously orphaned slots — those that were unlinked by *deleteNode()* but still occupied physical space — are now overwritten and made available. The invariant that valid data occupies indices 0 to *freeIndex-1* is fully restored.

The *maybeAutoGC()* function provides the automatic trigger mechanism. It maintains a module-level counter (*gcCounter*) that is incremented on every call. When the counter reaches 10, or when *freeIndex* has reached *MAX_LINES* (no more free slots), a garbage collection pass is inflated automatically. The counter resets to zero at the end of every GC pass, whether manual or automatic.

4.4 ui.c — ncurses User Interface

The `ui.c` module encapsulates all ncurses interactions. This separation ensures that no other module imports ncurses directly, all terminal operations are routed through the `ui` layer.

`ui_init()` initializes the ncurses session with `initscr()`, `raw()`, `noecho()`, `keypad(stdscr, TRUE)` and `curs_set(1)`. The call to `raw()` is preferred over `cbreak()` because it also disables signal character processing. `ui_cleanup()` calls `endwin()` to restore the terminal on exit.

`print_buffer()` is responsible for the entire screen render cycle. It saves the current cursor position with `getyx()`, clears the text area, redraws the bold header bar on row 0, draws a full-width dashed separator on row 1, then iterates through the linked list from `head`, printing each statement starting at `UI_TEXT_START_ROW` (row 2). The currently selected row is highlighted using a green reverse-video color pair (`COLOR_SELECTED = 1`). After drawing, the cursor is clamped to the valid text area and `refresh()` is called.

`cursorLine()` converts the current Y screen coordinate to a buffer index by calling `cursorLineNth()` to get the visual row offset, then walking the linked list that many steps from `head` and returning the actual `textbuffer[]` index. `cursorChar()` clamps the X coordinate to the length of the selected line and returns it as a character offset.

`ui_move_up()`, `ui_move_down()`, `ui_move_left()` and `ui_move_right()` each read the current cursor position, validate the movement against the buffer bounds, call `move()` to reposition the cursor and refresh the display. `ui_status()` writes a message to the last screen row using `A_REVERSE` styling.

4.5 main.c — Command Loop and ncurses Integration

The main entry point initializes the ncurses environment with `ui_init()`, `print_buffer()` and `keypad(stdscr, TRUE)`, which enables raw character input, suppresses echo and allows function keys (including arrow keys) to be captured as single key codes and render the initial screen. The main loop calls `getch()` and dispatches on the returned character.

The E command temporarily disables reverse-video styling and uses plain `mvprintw` at the bottom row to prompt a filename, then reads it with `getnstr()` to avoid overwriting the header bar. After loading `print_buffer()` redraws the screen and the cursor is moved to `UI_TEXT_START_ROW`.

Each command is mapped to its corresponding editing function. The I and D commands additionally call `cursorLine()` to obtain the buffer index of the selected line, invoke the corresponding editing function, call `maybeAutoGC()` to enforce the automatic GC policy and then refresh the display with `print_buffer()`. The R command similarly calls `cursorLine()` and `cursorChar()`. The G command calls `garbageCollection()` directly and displays the count of remaining active lines in the status bar.

5. Garbage Collection — Detailed Analysis

5.1 The Problem of Fragmentation

When a node is deleted from the linked list, its slot in `textbuffer[]` is not immediately reclaimed. The slot persists in the array, occupying one of the 100 available positions, but is no longer reachable by traversing the chain from `head`. This is analogous to heap fragmentation: free memory exists, but it is interleaved with live data and cannot be trivially reused.

As insertions continue, new nodes are always placed at `textbuffer[freeIndex]`, which advances monotonically. Without garbage collection, this pointer could reach 100 even if the logical list contains far fewer than 100 lines, because deleted slots are not recounted. At that point, no new insertions would be possible.

5.2 The Collection Algorithm

The implemented garbage collection algorithm is a mark-and-compact strategy analogous to the compaction phase used in some generational garbage collectors. It can be described in four steps:

1. **Traversal** — Walk the linked list from `head` using next pointers, copying each reachable statement into the temporary array `temp[]`.
2. **Reset** — Call `clearBuffer()` to zero all global state, including `head`, `tail`, `freeIndex` and all slots.
3. **Reconstruction** — Copy the saved statements back into `textbuffer[0...count-1]` with new sequential links.
4. **Pointer Update** — Set `head = 0`, `tail = count`, `freeIndex = count`.

The time complexity of this algorithm is $O(n)$ where n is the number of valid lines, since both the traversal and reconstruction phases make a single linear pass. The space overhead is one additional array of 100 node structures on the stack (node `temp [MAX_LINES]`), which is bounded and deterministic.

5.3 Buffer State Example

The following table illustrates the state of `textbuffer[]` after the example sequence from the project specification: load “aaa”, “bbb”, “ccc”, “ddd”; insert “xxx” before “ddd”, delete “bbb”. The table shows the pre-GC state, after which garbage collection produces a clean sequential layout.

Idx	Ptr	statement	prev	next	Notes
0	head	aaa	-1	1	Valid
1		bbb	0	2	Deleted — unlinked
2		ccc	0	3	Valid
3		ddd	2	4	Valid
4	tail	xxx	3	-1	Valid
5	free		-1	-1	Free slot

After `garbageCollection()` runs, the four valid lines are rearranged to indices 0-3, the head pointer is reset to 0, tail to 3 and `freeIndex` to 4. The slot at physical index 4 (formerly "xxx") is now free.

```
textbuffer[0] = { "aaa", prev=-1, next=1 } // head
textbuffer[1] = { "ccc", prev= 0, next=2 }
textbuffer[2] = { "ddd", prev= 1, next=3 }
textbuffer[3] = { "xxx", prev= 2, next=-1 } // tail
freeIndex = 4
```

5.4 Trigger Policy

Two trigger policy conditions are implemented in `maybeAutoGC()`:

- **Counter-Based:** After every 10 insert or delete operations, a GC pass is initiated regardless of the current fill level. This prevents excessive fragmentation from accumulating over long editing sessions.

- **Capacity-Based:** If `freeIndex` reaches `MAX_LINES(100)`, meaning no physical slots remain for new insertions, a GC pass is triggered immediately before the failure would otherwise occur. If the logical list is full (all 100 slots are occupied by reachable lines), the editor correctly reports that the buffer is full even after GC.

The user may also trigger GC manually at any time by pressing the G key, which is useful for performance-sensitive scenarios or for inspecting the compacted buffer state during testing.

6. User Interface

6.1 Screen Layout

The editor uses a three-zone screen layout managed entirely by ui.c:

- ◆ Row 0 — Header bar: bold command reference
- ◆ Row 1 — Separator: full width dashed line drawn with mvaddch.
- ◆ Rows 2+ — Text area: one line of the file per row, starting at UI_TEXT_START_ROW = 2.
- ◆ Last Row — Status Bar: reverse-video message area updated by ui_status().

6.2 ncurses Initialization

The ncurses library is initialized in main() with the following sequence:

```
initscr();          /* initialize the ncurses screen */
raw();              /* disable line buffering      */
noecho();           /* do not echo typed characters */
keypad(stdscr, TRUE); /* enable function / arrow keys */
curs_set(1);        /* show the cursor.            */
```

The call to raw() is preferred over cbreak() because it also disables signal character processing (Ctrl + C, Ctrl + Z) giving the application full control over all input. The call to keypad(stdscr, TRUE) is essential for capturing arrow key sequences, which ncurses translates into the constants KEY_UP, KEY_DOWN, KEY_LEFT and KEY_RIGHT that are handled in the switch statement.

6.2 Command Summary

Key	Command	Description
E	Edit	Prompts for a filename; loads the file into the text buffer
I	Insert	Inserts a new line after the cursor-selected line
D	Delete	Removes the cursor-selected line from the buffer
R	Replace	Overwrites the character at the cursor position
P	Print	Refreshes the screen display from head to tail
S	Save	Writes the current buffer back to the open file
G	Garbage Collect	Manually triggers a buffer compaction pass
Q	Quit	Closes the ncurses session and exits the program
↑ / ↓	Navigate lines	Moves the cursor up or down one line
← / →	Navigate chars	Moves the cursor left or right one character (used with R)
Enter	Select line	Confirms the cursor line for editing

7. Build System

7.1 Directory Structure

```

|_ TextEditor/
|_| include/
|_|_| editor.h
|_|_| operations.h
|_|_| garbageCollection.h
|_|_| ui.h
|_| src/
|_|_| main.c
|_|_| editor.c
|_|_| operations.c
|_|_| garbageCollection.c
|_|_| ui.c
|_| build/  <- Object files
|_| test/
|_|_| test1.txt
|_|_| test2.txt
|_| Makefile
|_| README.md

```

7.2 Makefile

The Makefile compiles each .c file in src/ to a corresponding .o object file in build/, then links all object files together with the ncurses library:

```
gcc
CFLAGS = -Wall -g -I include
LIBS   = -lncurses
TARGET = editor
BUILD  = build

SRCS = src/main.c src/editor.c src/operations.c \
      src/garbageCollection.c src/ui.c

OBJS = $(patsubst src/%.c, $(BUILD)/%.o, $(SRCS))

all: $(BUILD) $(BUILD)/$(TARGET)

$(BUILD):
    mkdir -p $(BUILD)

$(BUILD)/$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $(BUILD)/$(TARGET) $(OBJS) $(LIBS)

$(BUILD)/%.o: src/%.c
    $(CC) $(CFLAGS) -c $< -o $@

run: all
    ./$(BUILD)/$(TARGET)

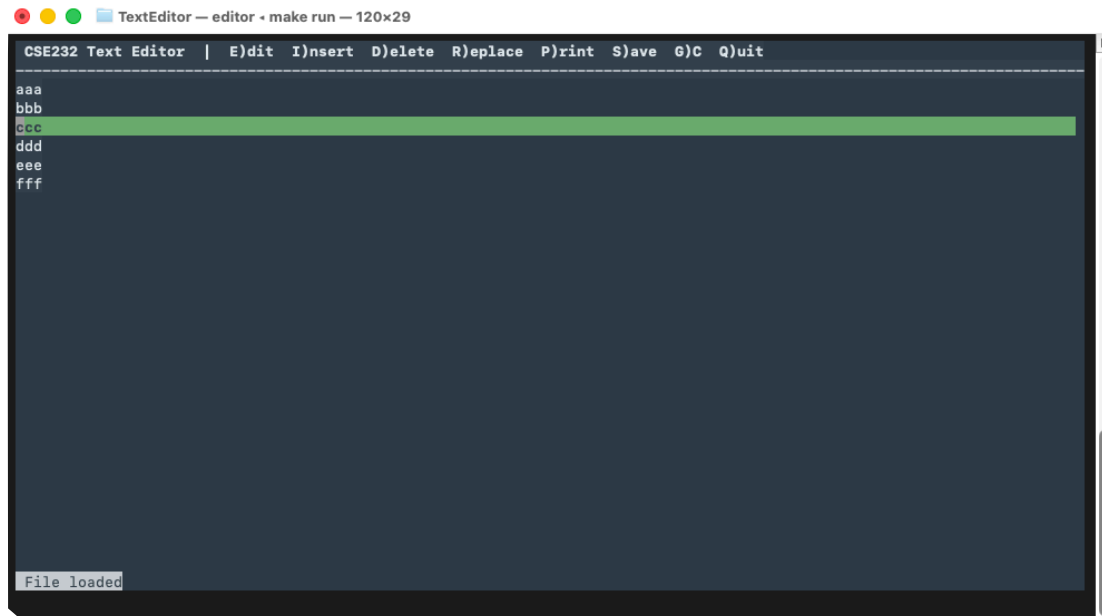
clean:
    rm -rf $(BUILD)
```

The `-Wall` flag enables all compiler warnings. The `-g` flag includes debug symbols, enabling gdb-assisted debugging. The `-I` include flag makes all header files accessible without path prefixes in source files.

8. Test Cases and Expected Behavior

8.1 Load and Save

Given the file mytext.txt containing four lines ("aaa", "bbb", "ccc", "ddd"), the E mytext.txt command should result in head = 0, tail = 3 and freeIndex = 4, with each nodes prev and next set to the adjacent index or NIL at the boundaries. Issuing S immediately after E should produce a file identical to the original.

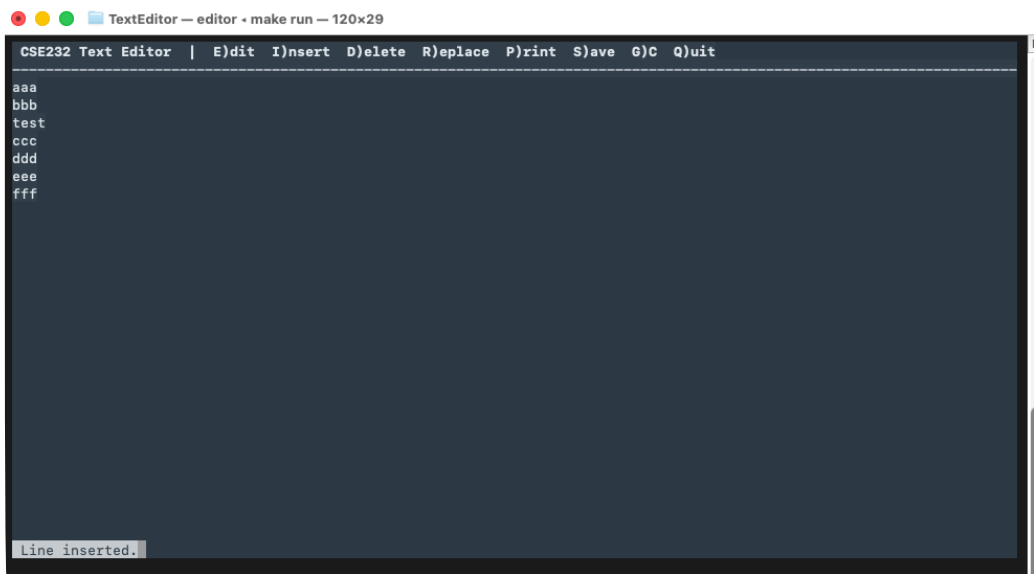


```

CSE232 Text Editor | E)dit I)nsert D)delete R)eplace P)rint S)ave G)C Q)uit
-----
aaa
bbb
ccc
ddd
eee
fff
File loaded
```

8.2 Insertion

After loading the four-line and inserting "xxx" before line at index 3 ("ddd"), the logical order should read "aaa" → "bbb" → "ccc" → "xxx" → "ddd". The new node occupies textbuffer[4] and freeIndex advances to 5. The tail pointer remains at 3 (the physical slot of "ddd"), since "ddd" was not moved, only its prev link was updated to point to 4

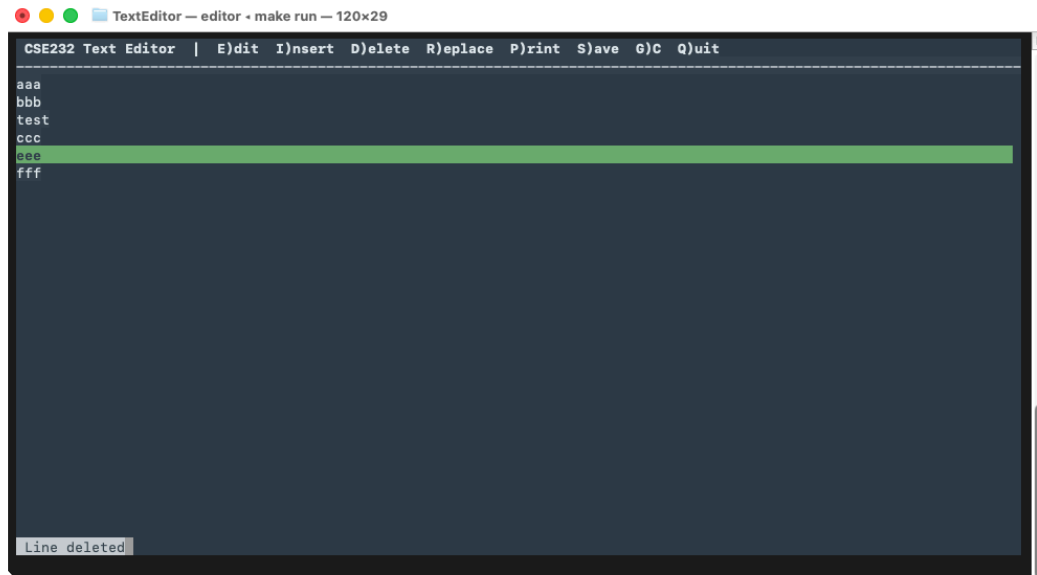


```

CSE232 Text Editor | E)dit I)nsert D)delete R)eplace P)rint S)ave G)C Q)uit
-----
aaa
bbb
test
ccc
ddd
eee
fff
Line inserted.
```

8.3 Deletion

Deleting the line at logical position 1 (“bbb”) should update textbuffer[0].next to skip over index 1 and point directly to index 2. The node at index 1 retains its statement content but its prev and next fields become NIL, marking it as orphaned. The logical traversal now yields “aaa” → “ccc” → “xxx” → “ddd”.

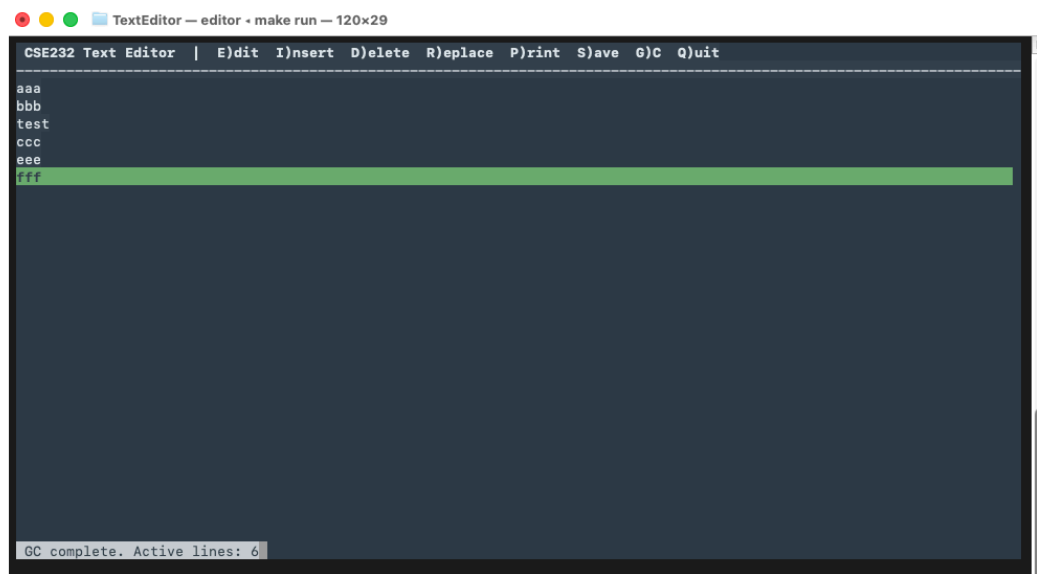


```
CSE232 Text Editor | E)dit I)nsert D)delete R)eplace P)rint S)ave G)C Q)uit
aaa
bbb
test
ccc
ddd
fff
Line deleted
```

8.4 Garbage Collection

After the operations above, four valid lines occupy physical indices 0, 2, 3 and 4 (index 1 is orphaned). Running garbageCollection() copies those four statements to temp[0..3], clears the buffer and rewrites them sequentially to textbuffer[0..3] with links 0 ↔ 1 ↔ 2 ↔ 3. The result matches the canonical post-GC state shown in the project specification.

```
textbuffer[0] = { "aaa", prev=-1, next=1 } // head
textbuffer[1] = { "ccc", prev= 0, next=2 }
textbuffer[2] = { "ddd", prev= 1, next=3 }
textbuffer[3] = { "xxx", prev= 2, next=-1 } // tail
freeIndex = 4
```



```
CSE232 Text Editor | E)dit I)nsert D)delete R)eplace P)rint S)ave G)C Q)uit
aaa
bbb
test
ccc
ddd
fff
GC complete. Active lines: 6
```

8.5 Automatic GC Trigger

A sequence of 10 consecutive delete or insert operations should automatically trigger `garbageCollection()` via `maybeAutoGC()` on the 10th operation. Verification can be performed by printing the `textbuffer` state before and after the 10th operation and confirming that `freeIndex` decreases to match the number of remaining valid lines.

9. Team Contributions and Responsibilities

The project was developed collaboratively, with each team member taking ownership of a specified subsystem. Responsibilities were clearly divided to ensure modular development, parallel progress and seamless integration. Below is a detailed breakdown of individual contributions.

Member 1 - Atakan Naktürk

- ✖ Data Structure & Core Engine
- ✖ editor.h and editor.c (clearBuffer, stripNewline, edit, save, global state)

Member 2 - Ferhat Canşı

- ✖ Editing Operations
- ✖ operations.h and operations.c (insert, deleteNode, replace, nthNode helper)

Member 3 - Musab Tunç

- ✖ User interface (GUI | TUI), ncurses UI and Cursor Mangement
- ✖ ui.h and ui.c (ncurses cursor movement, print_buffer, cursorLine, cursorChar)

Member 4 - Cihat Öztürk

- ✖ Garbage Collector
- ✖ garbageCollection.h and garbageCollection.c (garbageCollection, maybeAutoGC)

Member 5 - Ege Huriel

- ✖ Integration and Report
- ✖ main.c (command loop, keymapping), Makefile, Project Report

10. Conclusion

This project successfully implements the core functionality of a memory-managed text editor in C, demonstrating the practical application of several fundamental systems programming concepts: static memory management via array-backed linked lists, lazy deletion with deferred compaction, modular C program architecture with header/source separation and terminal user interface programming with the ncurses library.

The garbage collection component in particular illustrates an important systems programming trade-off: by deferring the cost of reclamation and batching it into periodic compaction passes, the editor avoids the per-operation overhead that would be incurred by maintaining a separate free list or by shifting array contents on every deletion. The automatic trigger mechanism (every 10 operations or an buffer exhaustion) ensures that compaction overhead is distributed evenly across the editing session rather than concentrated in any single operation.

The separation of concerns across `editor.c`, `operations.c`, `garbageCollection.c` and the `ui.c` reflects an intentional design decision: each module can be developed, tested and reasoned about independently, with the data structure layer (`editor.c`) serving as the sole owner of global mutable state. This architecture would scale naturally to a larger feature set and the remaining work on the ncurses UI layer is a contained, well scoped task that does not require changes to any other module.

References

1. CSE 232 Systems Programming Lecture Slides and Notes — Yeditepe University, 2026
2. Stack Overflow, Question on GarbageCollection, UI, Keymapping <https://stackoverflow.com>
3. Terminal Based TextEditor — <https://github.com/curlpipe/ox>
4. Terminal Based TextEditor w/ Python — <https://github.com/seeM/editor/blob/master/editor.py>
5. Debugging and Error Handling — ChatGPT & StackOverflow
6. CSE 232 Systems Programming - Term Project Instructions PDF — Yeditepe University, 2026
7. Ncurses - <https://bidb.itu.edu.tr/seyir-defteri/blog/2013/09/06/ncurses>
8. Writing in ncurses — <https://invisible-island.net/ncurses/ncurses-intro.html>
9. https://www.sbarjatiya.com/notes_wiki/index.php/Using_ncurses_library_with_C